

Core NavKit Architecture Design Notes

State Vector Definition

All compile time state definitions - static polymorphism design principle. Simple `Segment` fundamental utility class template.

```
template <int I, int SZ>
struct Segment {
    static constexpr int i = I;
    static constexpr int sz = SZ;
};
```

Helper utility functions to grab sub-selections of matrices and vectors

```
// only useful for grabbing square sub-matrix on diagonal
template <typename TSeg, typename TMat>
auto block(const TMat& m) {
    return m.template block<TSeg::sz, TSeg::sz>(TSeg::i, TSeg::i);
}

template <typename TSeg, typename TVec>
auto segment(const TVec& v) {
    return v.template segment<TSeg::sz>(TSeg::i);
}

// use case
Eigen::Vector<Scalar_t, StateDef::Pos::sz> d_pos = segment<StateDef::Pos>(x);
```

Example standard StateDef

```
struct InsStateDef {
    // standard navigation states
    using Pos      = Segment<0, 3>;
    using Vel      = Segment<3, 3>;
    using Att      = Segment<6, 3>;

    // IMU error states, bias and scale factor
    using GyroB    = Segment<9, 3>;
    using GyroSf   = Segment<12, 3>;
    using AccB     = Segment<15, 3>;
    using AccSf    = Segment<18, 3>;

    // size of the state vector
```

```
static constexpr int N = 21;
};
```

Tightly coupled GNSS StateDef

```
struct GnssTcStateDef {
    // standard navigation states
    using Pos      = Segment<0, 3>;
    using Vel      = Segment<3, 3>;
    using Att      = Segment<6, 3>;

    // IMU error states - bias and scale factor
    using GyroB    = Segment<9, 3>;
    using GyroSf   = Segment<12, 3>;
    using AccB     = Segment<15, 3>;
    using AccSf    = Segment<18, 3>;

    // GNSS receiver clock error states - bias and drift
    using ClkB     = Segment<21, 1>;
    using ClkD     = Segment<22, 1>;

    // size of the state vector
    static constexpr int N = 23;
};
```

Create a header that defines useful aliases and avoid circular dependencies

```
// Inside State.h
template <typename StateDef>
using State = Eigen::Vector<Scalar_t, StateDef::N>;

template <typename StateDef>
using StateCov = Eigen::Matrix<Scalar_t, StateDef::N, StateDef::N>;
```

Kalman Filter Concepts

Sensor Processing

The `Sensor` class is used to hold all configuration, and runtime and persistent state. The `SensorModel` class is used to provide all math functionality relevant to the Kalman filter. It uses the Curiously Recurring Template Pattern (CRTP) via `SensorModelBase` to effectively enforce a compile time static interface.

This is the basic layout: `Sensor<Model, BufferSize, NoisePolicy>`, which will be shown in detail below.

SensorModelBase class:

```

template <typename Derived, typename StateDef, int M_>
class SensorModelBase
{
public:
    static constexpr int M = M_;
    using State_t = State<StateDef>;

    using O_t = Eigen::Vector<Scalar_t, M>;
    using H_t = Eigen::Matrix<Scalar_t, M, StateDef::N>;
    using R_t = Eigen::Matrix<Scalar_t, M, M>;
    using K_t = Eigen::Matrix<Scalar_t, StateDef::N, M>;

    // NoiseContext provides whatever information is necessary to form measurement
    covariance R matrix
    static R_t compute_r(const typename Derived::NoiseContext& ctx)
    {
        return Derived::compute_r_impl(ctx);
    }

    static H_t compute_h(const State_t& x)
    {
        return Derived::compute_h_impl(x);
    }

    static O_t obs(const State_t& x)
    {
        return Derived::obs_impl(x);
    }
};

```

Simple measurement struct

```

template <int M>
struct Measurement
{
    Time_t time;
    Eigen::Vector<Scalar_t, M> z;
};

```

Sensor class that holds configuration and runtime information. It's effectively using the Policy-Based Design Pattern with the `Model` template type parameter.

```

struct DefaultNoisePolicy
{
    template <typename NoiseContext, typename Measurement>
    static void update(NoiseContext&, const Measurement&)
    {
        // default: do nothing
    }
};

```

```

    }
};

template <typename Model, size_t BufferSize, typename NoisePolicy =
DefaultNoisePolicy>
class Sensor
{
public:
    using Model_t = Model;
    using typename Model_t::O_t;
    using typename Model_t::H_t;
    using typename Model_t::R_t;
    using Measurement_t = Measurement<Model_t::M>;
    using NoiseContext_t = typename Model_t::NoiseContext;

    bool has_measurement() const
    {
        return !m_buffer.empty();
    }

    bool pop(Measurement_t& meas)
    {
        return m_buffer.pop(meas);
    }

    const NoiseContext_t& noise_context() const
    {
        return m_noise_ctx;
    }

    void update_noise_context(const Measurement_t& meas)
    {
        NoisePolicy::update(m_noise_ctx, meas);
    }

    // handle arbitrary state update after Navigator processes this Sensor
    void update()
    {
        // default no-op
    }

private:
    RingBuffer<Measurement_t, BufferSize> m_buffer;
    NoiseContext_t m_noise_ctx;
};

```

Example hypothetical noise policy for a GNSS sensor

```

struct GnssNoisePolicy
{
    template <typename NoiseContext, typename Measurement>
    static void update(NoiseContext& ctx, const Measurement& meas)

```

```

{
    // Example using measurement metadata
    ctx.sigma_h = meas.metadata.hdop * 1.5;
    ctx.sigma_v = meas.metadata.vdop * 2.0;
}
};

```

Simple loosely coupled GNSS measurement class example:

```

#include "State.h"

template <typename StateDef>
class GnssPosModel : public SensorModelBase<GnssPosModel<StateDef>, StateDef, 3>
{
public:
    using Base = SensorModelBase<GnssPosModel<StateDef>, StateDef, 3>;
    using typename Base::State_t;
    using typename Base::H_t;
    using typename Base::R_t;
    using typename Base::O_t;

    struct NoiseContext
    {
        float sigma_h;
        float sigma_v;
    };

    static H_t compute_h_impl(const State_t&)
    {
        // simple definition
        // H_t H = H_t::Zero();
        // H.block<M, M>(0, StateDef::Pos::i) = R_t::Identity();
        // return H;

        // static local initialization with lambda
        // if we want to support returning a "static const H_t&" in the future
        // this would be statically initialized once
        static H_t H = [] {
            H_t tmp = H_t::Zero();
            tmp.block<M, M>(0, StateDef::Pos::i) =
                Eigen::Matrix<Scalar_t, M, M>::Identity();
            return tmp;
        }();
        return H;
    }

    static R_t compute_r_impl(const NoiseContext& ctx)
    {
        R_t R = R_t::Zero();

        // simple diagonal NED measurement covariance example with only horizontal

```

```

and vertical sigma values
    R(0,0) = ctx.sigma_h * ctx.sigma_h;
    R(1,1) = ctx.sigma_h * ctx.sigma_h;
    R(2,2) = ctx.sigma_v * ctx.sigma_v;

    return R;
}

// observation function of whole state
static O_t obs_impl(const State_t& x)
{
    return segment<StateDef::Pos>(x);
    // or
    // return x.segment<StateDef::Pos::sz>(StateDef::Pos::i);
    // or whatever more complex non-linear observation logic would happen here
}
};

```

Example of creating `SensorModels`

```

// Model then Sensor
using GnssModel = GnssPosModel<StateDef>;
using GnssSensor = Sensor<GnssModel, 32, GnssNoisePolicy>;

// or one line Sensor
template <typename StateDef>
using GnssPosSensor = Sensor<GnssPosModel<StateDef>, 32, GnssNoisePolicy>;

// later creating a tuple of Sensors
using Sensors = std::tuple<GnssPosSensor<StateDef>, BaroSensor<StateDef>>;

// Other SensorModels
GnssPosModel<InsStateDef>
BaroModel<InsStateDef>
MagModel<InsStateDef>

```

Kalman Filter Core

Kalman filter class:

```

#include "State.h"

template <typename StateDef,
          typename InjectionPolicy,
          typename ResetPolicy = DefaultResetPolicy<StateDef>
          >
class KalmanFilter {
public:
    using State_t = State<StateDef>;

```

```

using P_t = StateCov<StateDef>;

// statically stored identity matrix for re-use
static const P_t& I()
{
    static P_t I = P_t::Identity();
    return I;
}

template <typename Model>
void covariance_update(
    const P_t& P_i,
    const typename Model::H_t& H,
    const typename Model::R_t& R,
    const typename Model::O_t& innovation,
    typename Model::R_t& S,
    typename Model::K_t& K,
    State_t& dx,
    P_t& P_f)
{
    // innovation covariance in observation space
    S = H * P_i * H.transpose() + R;

    // "solve()" instead of inverse for numerical stability
    K = P_i * H.transpose() * S.ldlt().solve(Model::R_t::Identity());

    // Kalman observation correction
    dx = K * innovation;

    // Joseph form posteriori covariance for numerical stability
    P_f = (I() - K * H) * P_i * (I() - K * H).transpose() + K * R *
K.transpose();
}

// "x" is whole state prior to observation (priori)
// "z" is measurement observation
template <typename Model>
void observation_update(const typename Model::O_t& z, const typename
Model::NoiseContext& ctx)
{
    // calculate innovation given non-linear measurement observation model
    typename Model::O_t innov = z - Model::obs(m_x);

    // update covariance given observation
    typename Model::R_t S;
    typename Model::K_t K;
    covariance_update<Model>(
        m_P_i,
        Model::compute_h(m_x),
        Model::compute_r(ctx),
        innov,
        S,
        K,
        m_dx,

```

```

        m_P_f
    );

    m_P_i = m_P_f;
}

// function template to process sensor
template <typename Sensor>
void process_sensor(Sensor& sensor)
{
    using Model = typename Sensor::Model_t;
    using typename Model::NoiseContext;
    using typename Sensor::Measurement_t;

    while (sensor.has_measurement()) {
        Measurement_t meas;
        if (!sensor.pop(meas)) {
            // TODO: handle errors
        }
        sensor.update_noise_context(meas);
        const NoiseContext& ctx = sensor.noise_context();

        observation_update<Model>(meas.z, ctx);
    }
}

void inject(State_t& x, const State_t& dx)
{
    InjectionPolicy::apply(x, dx);
}

void reset(State_t& x, State_t& dx, P_t& P)
{
    ResetPolicy::template reset_covariance<StateDef, State_t, P_t>(x, dx, P);
    ResetPolicy::reset_dx(dx);
}

private:
    State_t m_x;
    State_t m_dx;

    // member priori (i) and posteriori (f) covariance matrices
    P_t m_P_i;
    P_t m_P_f;
}

```

EKF Injection Policy

Injection policy building blocks:


```
template<typename Seg>
struct AdditiveInjection
{
    template<typename State_t>
    static void apply(State_t& x, const State_t& dx)
    {
        segment<Seg>(x) += segment<Seg>(dx);
    }
};
```

```
template<typename Seg>
struct QuaternionInjection
{
    template<typename State_t>
    static void apply(State_t& x, const State_t& dx)
    {
        auto dtheta = segment<Seg>(dx);
        Quaternion dq = small_angle_quat(dtheta);
        auto& q = segment<Seg>(x);

        q = dq * q;
        q.normalize();
    }
};
```

Core INS only injection policy as a reference. The user must create an **InjectionPolicy** that directly maps to the full **StateDef**.

```
template<typename StateDef>
struct InsInjectionPolicy
{
    template<typename State_t>
    static void apply(State_t& x, const State_t& dx)
    {
        AdditiveInjection<typename StateDef::Pos>::apply(x, dx);
        AdditiveInjection<typename StateDef::Vel>::apply(x, dx);
        QuaternionInjection<typename StateDef::Att>::apply(x, dx);
    }
};
```

Example InjectionPolicy for a **StateDef** that contains IMU error states:

```
template<typename StateDef>
struct ImuInsInjectionPolicy
{
    template<typename State_t>
```

```

static void apply(State_t& x, const State_t& dx)
{
    // standard navigation states
    AdditiveInjection<typename StateDef::Pos>::apply(x, dx);
    AdditiveInjection<typename StateDef::Vel>::apply(x, dx);
    QuaternionInjection<typename StateDef::Att>::apply(x, dx);

    // IMU error states
    AdditiveInjection<typename StateDef::GyroB>::apply(x, dx);
    AdditiveInjection<typename StateDef::GyroSf>::apply(x, dx);
    AdditiveInjection<typename StateDef::AccB>::apply(x, dx);
    AdditiveInjection<typename StateDef::AccSf>::apply(x, dx);
}
};

```

ResetPolicy defines how to reset the error state vector and adjust the covariance for the attitude error state.

```

template<typename StateDef>
struct DefaultResetPolicy
{
    template<typename State_t>
    static void reset_dx(State_t& dx)
    {
        dx.setZero();
    }

    template <typename StateDef, typename State_t, typename P_t>
    static void reset_covariance(
        const State_t& x,
        const State_t& dx,
        P_t& P)
    {
        // assumes error state is right-multiplicative quaternion with small-angle
        approx
        //  $G = I - 0.5 * \text{skew}(d\_theta)$ 
        //  $P = G P G^T$ 
        Eigen::Matrix3<Scalar_t> G_att =
            Eigen::Matrix3<Scalar_t>::Identity() - 0.5 *
            skew(segment<StateDef::Att>(dx));

        P_t G = P_t::Identity();
        block<StateDef::Att>(G) = G_att;

        P = G * P * G.transpose();
    }
};

```

Navigator

The navigator will own the **KalmanFilter** and all **Sensors**, and will orchestrate all main logic.

Navigator class:

```
template <typename StateDef, typename Sensors, typename Filter,
          typename UpdatePolicy = UpdatePostFilter>
class Navigator
{
public:
    static_assert(StateDef::N > 0, "State size must be positive");

    using Filter_t = Filter<StateDef>;

    void process_measurements()
    {
        /*
         * Compile time generic sensor processing of an `std::tuple<>` of `Sensor`s.
         * Uses `std::apply()` (C++17) to effectively iterate through the tuple container at
         * compile time.
         */
        std::apply(
            [this](auto&... sensor)
            {
                (
                    m_filter.process_sensor(sensor),
                    UpdatePolicy::sensor_update(m_filter, sensor),
                    ...
                );
            },
            m_sensors
        );

        UpdatePolicy::filter_update(m_filter);
    }

private:
    Filter_t m_filter{};
    Sensors m_sensors{};
};
```

Navigator Update Policy

```
template<typename Filter>
struct UpdatePostSensor
{
    template<typename Sensor>
    static void sensor_update(Filter& f, const Sensor& s)
    {
        s.update();
        f.inject();
        f.reset();
    }
};
```

```

    static void filter_update(Filter& f)
    {
        // no-op
    }
};

```

```

template<typename Filter>
struct UpdatePostFilter
{
    template<typename Sensor>
    static void sensor_update(Filter& f, const Sensor& s)
    {
        // no-op
    }

    static void filter_update(Filter& f)
    {
        f.inject();
        f.reset();
    }
};

```

Client Code

The client can use C++ header config files that define compile time types, aliases, and variables.

```

// Config.h

struct Config {
    using Scalar_t = float;
    static constexpr size_t IMU_BUFF_SIZE = 256;

    using StateDef = InsStateDef;

    template<typename Filter>
    using UpdatePolicy = UpdatePostFilter<Filter>;

    template <typename StateDef_t>
    using InjectionPolicy = InsInjectionPolicy<StateDef_t>;

    template <typename StateDef_t>
    using ResetPolicy = DefaultResetPolicy<StateDef_t>;

    template <typename StateDef_t>
    using Sensors = GnssPosBaroSensors<StateDef_t>;
};

```

Starting with an example of some concrete **Sensor** types, and how we can use those to create an aliased **Sensors** tuple that's configured at compile time.

Sensor assembly, which will have dedicated files per sensor tuple configuraiton.

```
// GnssPosBaroSensors.h

// define individual sensors
template <typename StateDef>
using GnssPosSensor = Sensor
<GnssPosModel<StateDef>, 256, GnssNoisePolicy>;

template <typename StateDef>
using BaroSensor = Sensor<BaroModel<StateDef>, 64>;

template <typename StateDef>
using GnssPosBaroSensors = std::tuple<
    GnssPosSensor<StateDef>,
    BaroSensor<StateDef>
>;
```

Navigation system assembly, which can be used by application (sim or firmware) code. Ideally this should remain unchanged regardless of the Config configuraiton.

```
// NavAssembly.h

using Config::StateDef;
using Config::Sensors;

// define filter
using Filter = KalmanFilter<
    StateDef,
    Config::InjectionPolicy<StateDef>,
    Config::ResetPolicy<StateDef>
>;

// define navigator type
using Nav = Navigator<StateDef, Sensors, Filter, Config::UpdatePolicy<Filter>>;

// instantiate navigator
Nav nav;
```

Common Utilities

Ring buffer class:

```
enum class OverflowPolicy
{
```

```
    Reject,
    OverwriteOldest
};

template <typename T, size_t N,
    OverflowPolicy policy = OverflowPolicy::OverwriteOldest>
class RingBuffer
{
public:
    bool push(const T& value)
    {
        if (m_count == N) {
            if constexpr (policy == OverflowPolicy::OverwriteOldest) {
                advance_tail();
            }
            else {
                return false;
            }
        }

        m_data[m_head] = value;

        m_head = (m_head + 1) % N;
        ++m_count;

        return true;
    }

    bool pop(T& out)
    {
        if (m_count == 0)
            return false;

        out = m_data[m_tail];

        advance_tail();

        return true;
    }

    bool empty() const
    {
        return m_count == 0;
    }

    size_t size() const
    {
        return m_count;
    }

    void clear()
    {
        m_head = 0;
    }
};
```

```

        m_tail = 0;
        m_count = 0;
    }

    bool front(T& out) const
    {
        if (m_count == 0)
            return false;

        out = m_data[m_tail];
        return true;
    }

private:
    void advance_tail()
    {
        m_tail = (m_tail + 1) % N;
        --m_count;
    }

    std::array<T, N> m_data{};

    size_t m_head{0};
    size_t m_tail{0};
    size_t m_count{0};
};

```

// Future TODO: add getters for **Sensors** tuple with various options like using type identifiers

Errors and Return Types

Here are some ideas for some basic return types.

Basic status return type class.

```

enum class Status {
    OK,
    NoData,
    Invalid,
    Overflow
};

// basic use case
Status read(ImuData& out);

```

Slightly more complex status return type option.

```

template <typename T>
struct Result {

```

```

    T value;
    Status status;
};

// basic use case
Result<ImuData> read(InputData& in)
{
    ImuData out{};
    if (in.is_empty()) {
        return Result<ImuData>{out, Status::NoData};
    }

    // do processing
    return Result<ImuData>{out, Status::OK};
}

```

Error types more specific to expected errors during filtering math procesing.

```

enum class NavStatus {
    OK,
    NumericalIssue,
    CovarianceNotPSD,
    BadMeasurement
};

```

NavStatus return type use case.

```

NavStatus updateGnss(const GnssMeas& z) {
    if (!isValid(z)) {
        return NavStatus::BadMeasurement;
    }

    // do update

    if (!P.allFinite()) {
        return NavStatus::NumericalIssue;
    }

    return NavStatus::OK;
}

```